



Lab Assignment – 6 (Database Automation: Procedures, Functions & Triggers)

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Objective: Master database automation. You are provided with a schema and a set of **Business Requirements**. You must design the Stored Procedures, Functions, and Triggers to satisfy these requirements without breaking data integrity.

Pre-Lab Setup: Run this block to initialize the University environment.

-- 0. Reset Environment

```
SET SQL_SAFE_UPDATES = 0;
DROP TRIGGER IF EXISTS before_insert_enrollment;
DROP TRIGGER IF EXISTS after_update_grade;
DROP PROCEDURE IF EXISTS Enroll_Student;
DROP PROCEDURE IF EXISTS Department_Report;
DROP FUNCTION IF EXISTS Calculate_GPA;
DROP TABLE IF EXISTS Grade_Log;
DROP TABLE IF EXISTS Enrollment;
DROP TABLE IF EXISTS Student;
DROP TABLE IF EXISTS Course;
```

-- 1. Create Schema

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Dept_Name VARCHAR(30),
    Total_Credits INT DEFAULT 0
);
```

```
CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50),
    Credits INT
);
```

```
CREATE TABLE Enrollment (
    Enroll_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    Grade CHAR(1),
```

```

FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);

CREATE TABLE Grade_Log (
  Log_ID INT AUTO_INCREMENT PRIMARY KEY,
  Student_ID INT,
  Old_Grade CHAR(1),
  New_Grade CHAR(1),
  Change_Date DATETIME
);

-- 2. Insert Sample Data
INSERT INTO Student VALUES (1, 'Aarav', 'CSE', 0), (2, 'Neha', 'CSE', 0), (3, 'Rohan', 'Physics', 0);
INSERT INTO Course VALUES (101, 'DBMS', 4), (102, 'OS', 3), (103, 'Mathematics', 4);

-- Initial Enrollments
INSERT INTO Enrollment VALUES (1, 1, 101, 'A'), (2, 1, 102, 'B'), (3, 2, 101, 'A');

UPDATE Student SET Total_Credits = 7 WHERE Student_ID = 1;
UPDATE Student SET Total_Credits = 4 WHERE Student_ID = 2;

```

Q 1: Enrollment Automation

The University requires a streamlined enrollment process that keeps student records synchronized.

- Create a procedure `Enroll_Student` (Inputs: Student ID, Course ID, Grade).
- When a student is enrolled in a course, the system must **automatically update** that student's `Total_Credits` in the `Student` table to reflect the additional credit load of the new course.
- Enroll **Rohan** (Student 3) in **OS** (Course 102) with a 'B' grade. Check that the system updated his credits correctly.

Q 2: Grading Standard

The Exam Cell requires a dynamic GPA calculator for report generation.

- Create a function `Calculate_GPA` that takes a Student ID and returns their GPA.
- **Grading Scale:** A=10, B=8, C=6, D=4, F=0.
- **Formula:** $\text{Weighted Average} = \frac{\text{sum}(\text{GradePoints} * \text{Credits})}{\text{sum}(\text{Credits})}$.
- Generate a list of all Students and their current GPA.

Q 3: Integrity & Auditing

The database must enforce strict data quality rules that the application layer might miss.

- It is illegal to enroll a student without a valid grade. Create a Trigger to **prevent** any insertion into the Enrollment table if the Grade is NULL. The database must reject the transaction with a custom error message.
- Grade tampering is a security risk. Create a Trigger that keeps a permanent history of **grade changes** in the Grade_Log table. It must record who was changed, what the old grade was, what the new grade is, and when it happened.
- **Check:**
 1. Attempt to insert a NULL grade.
 2. Change Aarav's DBMS grade from 'A' to 'B' and show the log entry.

Q 4: The HOD Report

- The Head of Department needs a one-step command to generate a performance report for any specific department.
- A summary table showing the Department Name, Total Student Count, and the **Average GPA** of students in that department.
- You must reuse the Calculate_GPA function from Q2 inside this report.

-- 1. STORED PROCEDURE STRUCTURE

```
DELIMITER //
CREATE PROCEDURE Procedure_Name(IN param1 INT, IN param2 CHAR(1))
BEGIN
    -- You can run standard SQL queries here (INSERT, UPDATE, DELETE).

END //
DELIMITER ;
```

-- 2. FUNCTION STRUCTURE

```
DELIMITER //
CREATE FUNCTION Function_Name(param1 <datatype>)
RETURNS <datatype>
DETERMINISTIC
BEGIN

END //
DELIMITER ;
```

-- 3. TRIGGER STRUCTURE

```
DELIMITER //
CREATE TRIGGER Trigger_Name
BEFORE INSERT -- Or: AFTER UPDATE
ON Table_Name
FOR EACH ROW
BEGIN
    -- [Hint]: 'NEW.ColumnName' refers to the data being inserted/updated.
    -- [Hint]: 'OLD.ColumnName' refers to the data that existed before the update.
    -- [Logic]: Write your IF conditions or INSERT statements here.
```

```
END //
DELIMITER ;
```

-- 4. RAISING AN ERROR (For Validation)

-- Use this command inside a Trigger to stop a bad transaction:

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Your Error Message Here';
```